Session 1 Summary (1)

```
SELECT
  col,
  col_expression AS alias
FROM table
WHERE conds
ORDER BY col/alias DESC
LIMIT # OFFSET #_skip;
```

conds: LIKE, GLOB, etc.

```
SELECT
  OrderID,
  ProductID,
  UnitPrice * Quantity AS TotalPrice
FROM OrderDetails
WHERE TotalPrice > 12000
ORDER BY TotalPrice DESC;

* In Standard SQL
WHERE
  UnitPrice * Quantity > 12000
```

Session 1 Summary (2)

```
SELECT
  col_1,
  AGG(col_2/exp) AS alias
FROM table
WHERE conds
GROUP BY col_1
ORDER BY col/alias DESC
LIMIT # OFFSET #_skip;

AGG(): MIN(), MAX(), etc.
```

```
SELECT
  ContactTitle,
  COUNT(*) AS TotalByContactTitle
FROM Customers
GROUP BY ContactTitle
ORDER BY TotalByContactTitle DESC;
```

Today (1)

```
SELECT
  col_1,
  AGG(col_2/exp) AS alias
FROM table
WHERE conds
GROUP BY col_1
HAVING conds
ORDER BY col/alias DESC
LIMIT # OFFSET #_skip;

+Subquery: query within
query
```

Today (2)

```
SELECT
  col_1,
  AGG(col_2/exp) AS alias
FROM table_1
  JOIN table_2 ON conds
WHERE conds
GROUP BY col_1
HAVING conds
ORDER BY col/alias DESC
LIMIT # OFFSET #_skip;

+variants of JOIN
```

Session 2 Summary (1)

```
SELECT
  col_1,
  AGG(col_2/exp) AS alias
FROM table
WHERE conds
GROUP BY col_1
HAVING conds
ORDER BY col/alias DESC
LIMIT # OFFSET #_skip;
```

+ “Bare Columns”

```
SELECT
  OrderID,
  SUM(UnitPrice * Quantity) AS
    TotalOrderValue
FROM OrderDetails
GROUP BY OrderID
HAVING TotalOrderValue > 12000
ORDER BY TotalOrderValue DESC;

* In Standard SQL
HAVING
  SUM(UnitPrice * Quantity) > 12000
```

Session 2 Summary (2)

```
SELECT
  ShipCountry,
  AVG(Freight) AS AverageFreight
FROM Orders
WHERE OrderDate > DATETIME(
  (SELECT MAX(OrderDate) FROM Orders), "-1 year"
)
GROUP BY ShipCountry
ORDER BY AverageFreight DESC
LIMIT 3;
```

Session 2 Summary (3)

```
SELECT
  col_1,
  AGG(col_2/exp) AS alias
FROM table_1
  JOIN table_2 ON conds
WHERE conds
GROUP BY col_1
HAVING conds
ORDER BY col/alias DESC
LIMIT # OFFSET #_skip;
```

+variants of JOIN

```
SELECT
  Customers.CustomerID,
  CompanyName
FROM Customers
  LEFT JOIN Orders
    ON Customers.CustomerID =
      Orders.CustomerID
WHERE Orders.CustomerID IS NULL;
```

Session 2 Summary (4)

```
SELECT Customers.CustomerID, CompanyName
FROM Customers
   LEFT JOIN (
      SELECT CustomerID, EmployeeID
      FROM Orders
      WHERE EmployeeID = 4
   ) AS marg_orders
   ON Customers.CustomerID = marg_orders.CustomerID
WHERE marg_orders.CustomerID IS NULL;
```

Today

- More on the Margaret ex.
- CTE and temporary table
- Self-join
- A glance of window functions
- CASE keyword
- UNION keyword

Margaret Exercise

- Find customers that never ordered from Margaret (EmployeeID=4)

Customers

CustomerID	CompanyName
1	Company 1
2	Company 2
3	Company 3

Orders

OrderID	CustomerID	EmployeeID
1	1	1
2	1	4
3	2	3

Margaret Ex – Left Join + Subquery (1)

Customers

CustomerID	CompanyName
1	Company 1
2	Company 2
3	Company 3

Orders

OrderID	CustomerID	EmployeeID
1	1	1
2	1	4
3	2	3

```
SELECT Customers.CustomerID, CompanyName
FROM Customers
  LEFT JOIN (
    SELECT OrderID, CustomerID, EmployeeID
    FROM Orders
    WHERE EmployeeID = 4
  ) AS MargaretOrders
  ON Customers.CustomerID = MargaretOrders.CustomerID
WHERE OrderID IS NULL;
```

MargaretOrders

OrderID	CustomerID	EmployeeID
2	1	4

Margaret Ex – Left Join + Subquery (2)

Customers

CustomerID	CompanyName
1	Company 1
2	Company 2
3	Company 3

MargaretOrders

OrderID	CustomerID	EmployeeID
2	1	4

```
SELECT Customers.CustomerID, CompanyName
FROM Customers
  LEFT JOIN (
    SELECT OrderID, CustomerID, EmployeeID
    FROM Orders
    WHERE EmployeeID = 4
  ) AS MargaretOrders
  ON Customers.CustomerID = MargaretOrders.CustomerID
WHERE OrderID IS NULL;
```

Customers. CustomerID	CompanyName	OrderID	MargaretOrders. CustomerID	EmployeeID
1	Company 1	2	1	4
✓ 2	Company 2	Null	Null	Null
✓ 3	Company 3	Null	Null	Null

Margaret Ex – Left Join with Extra Cond.

Customers

CustomerID	CompanyName
1	Company 1
2	Company 2
3	Company 3

Orders

OrderID	CustomerID	EmployeeID
1	1	1
2	1	4
3	2	3

```
SELECT Customers.CustomerID, CompanyName
FROM Customers
  LEFT JOIN Orders
  ON Customers.CustomerID = Orders.CustomerID
     AND Orders.EmployeeID = 4
WHERE Orders.CustomerID IS NULL;
```

	Customers. CustomerID	CompanyName	OrderID	Orders. CustomerID	EmployeeID
	1	Company 1	2	1	4
✓	2	Company 2	Null	Null	Null
✓	3	Company 3	Null	Null	Null

Margaret Ex – Left Join First, and then…???

Customers

CustomerID	CompanyName
1	Company 1
2	Company 2
3	Company 3

Orders

OrderID	CustomerID	EmployeeID
1	1	1
2	1	4
3	2	3

```
SELECT Customers.CustomerID, CompanyName
FROM Customers
  LEFT JOIN Orders
  ON Customers.CustomerID = Orders.CustomerID
WHERE EmployeeID != 4;
```

	Customers. CustomerID	CompanyName	OrderID	Orders. CustomerID	EmployeeID
✓	1	Company 1	1	1	1
	1	Company 1	2	1	4
✓	2	Company 2	3	2	3
✓	3	Company 3	Null	Null	Null

Margaret Ex – Two More Methods (Set Op.)

```sql
-- WHERE, NOT IN, and subquery
SELECT CustomerID, CompanyName
FROM Customers
WHERE CustomerID NOT IN
   (SELECT CustomerID FROM Orders WHERE EmployeeID = 4);

-- EXCEPT keyword (Note: CompanyName cannot be there)
SELECT CustomerID
FROM Customers
EXCEPT
SELECT CustomerID
FROM Orders
WHERE EmployeeID = 4;
```

Performance Optimization

- A great follow-up question!
- So,…
- In this ex.,…

Common Sense Optimization

- Do not use `SELECT *`
- Filter early with `WHERE` instead of `HAVING` after `GROUP BY`
- Index fields/columns frequently found in `JOIN`, `WHERE`, `ORDER BY`, and `GROUP BY` clauses
- Don’t over index (as it slows down `INSERT`, `UPDATE`, `DELETE` Operations)
- …